```
import React, { useEffect, useMemo, useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import CaseClientReportsTab from '../reports/CaseClientReportsTab';
import {
  FileText,
  Upload,
  Trash2,
  CheckSquare,
  Calendar as CalendarIcon,
  DollarSign,
  Clock,
  Edit,
  Check,
  Eye,
  ArrowLeft,
  Plus,
  Receipt,
} from 'lucide-react';

import { getAuditForCase, AuditLogItem } from '../../services/auditService';
import { getCaseById, CaseData, updateCase } from '../../services/caseService';
import {
  getTasksForCase,
  addTaskToCase,
  updateTask,
  deleteTask,
  TaskData,
  submitTaskForApproval,
  approveTask,
  rejectTask,
} from '../../services/taskService';
import {
  getEventsForCase,
  addEventToCase,
  updateEvent,
  deleteEvent,
  CaseEvent,
} from '../../services/eventService';
import {
  getDocumentsForCase,
  addDocumentToCase,
  deleteDocument,
  CaseDocument,
} from '../../services/documentService';
```

```
import {
  getInvoicesForCase,
  addInvoiceToCase,
  uploadProof,
  uploadInvoiceFile,
  deleteInvoice,
  Invoice,
} from '../../services/invoiceService';

const BACKEND_URL = import.meta.env.VITE_BACKEND_URL || 'http://localhost:5000';
const API_URL = import.meta.env.VITE_BACKEND_API_URL || 'http://localhost:5000/api';
const getToken = () => localStorage.getItem('token');

type StaffUser = { _id: string; name: string; email: string; role: string };

type NewDocForm = {
  name: string;
  file?: File;
};

const eventTypes = ['Deadline', 'Court', 'Meeting', 'Other'];


const STAGE_ORDER = [
  'On Boarding',
  'Under Submission',
  'Pre trial',
  'Mediation',
  'Hearing',
  'Appeal',
  'Pronouncement',
  'Cope of Judgement',
  'Execution',
  'Closed',
];

interface CaseWorkspaceProps {
  userRole: string;
}

const formatRwf = (value: number) => `RWF ${Math.round(value).toLocaleString('en-US')}`;

const parseBudgetToNumber = (budget: unknown): number => {
  if (budget === null || budget === undefined) return 0;
```

```
  if (typeof budget === 'number') return budget;
  const cleaned = String(budget).replace(/[^\d.]/g, '');
  const n = Number(cleaned);
  return Number.isFinite(n) ? n : 0;
};

const getInvoiceStatusChip = (status: Invoice['status']) => {
  return status === 'Paid' ? 'bg-green-50 text-green-700' : 'bg-yellow-50 text-yellow-700';
};

const CaseWorkspace: React.FC<CaseWorkspaceProps> = ({ userRole }) => {
  const { id } = useParams();
  const navigate = useNavigate();

  const isAssociate = ['associate', 'lawyer', 'intern'].includes(userRole);
  const canManageCase = userRole === 'managing_director' || userRole ===
'executive_assistant';
  const canAssignTasks = userRole === 'managing_director' || userRole ===
'executive_assistant';
  const canManageBilling = userRole === 'managing_director' || userRole ===
'executive_assistant';
  const canManageCalendar = userRole === 'managing_director' || userRole ===
'executive_assistant';
  const canManageDocuments = userRole === 'managing_director' || userRole ===
'executive_assistant';

  // Case data
  const [caseData, setCaseData] = useState<CaseData | null>(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  // Tabs
  const [activeTab, setActiveTab] = useState<
    'overview' | 'tasks' | 'calendar' | 'documents' | 'billing' | 'audit' | 'reports'
  >('overview');

  // Staff list
  const [staffUsers, setStaffUsers] = useState<StaffUser[]>([]);
  const [staffLoading, setStaffLoading] = useState(false);
  const [staffError, setStaffError] = useState('');

  // Tasks
  const [tasks, setTasks] = useState<TaskData[]>([]);
  const [tasksLoading, setTasksLoading] = useState(false);
```

```
const [tasksError, setTasksError] = useState('');
const [showAddTask, setShowAddTask] = useState(false);
const [showEditTask, setShowEditTask] = useState(false);
const [editTask, setEditTask] = useState<TaskData | null>(null);
const [approvalComment, setApprovalComment] = useState('');
const [approvalLoading, setApprovalLoading] = useState(false);

const [newTask, setNewTask] = useState<
  Omit<TaskData, '_id' | 'caseId' | 'createdAt' | 'updatedAt'> & { description?: string }
>({
  title: '',
  priority: 'Medium',
  status: 'Not Started',
  assignee: '',
  dueDate: '',
  description: '',
  requiresApproval: false,
});

const handleSubmitForApproval = async () => {
  if (!editTask?._id) return;
  try {
    setApprovalLoading(true);
    await submitTaskForApproval(editTask._id);
    await reloadTasks();
    setShowEditTask(false);
    setEditTask(null);
  } catch (err: any) {
    setTasksError(err.message || 'Failed to submit for approval');
  } finally {
    setApprovalLoading(false);
  }
};

const handleApprove = async () => {
  if (!editTask?._id) return;
  try {
    setApprovalLoading(true);
    await approveTask(editTask._id, approvalComment);
    await reloadTasks();
    setShowEditTask(false);
    setEditTask(null);
    setApprovalComment('');
  } catch (err: any) {
```

```
      setTasksError(err.message || 'Failed to approve task');
    } finally {
      setApprovalLoading(false);
    }
  };

  const handleReject = async () => {
    if (!editTask?._id) return;
    try {
      setApprovalLoading(true);
      await rejectTask(editTask._id, approvalComment);
      await reloadTasks();
      setShowEditTask(false);
      setEditTask(null);
      setApprovalComment('');
    } catch (err: any) {
      setTasksError(err.message || 'Failed to reject task');
    } finally {
      setApprovalLoading(false);
    }
  };

  // Events
  const [events, setEvents] = useState<CaseEvent[]>([]);
  const [showAddEvent, setShowAddEvent] = useState(false);
  const [showEditEvent, setShowEditEvent] = useState(false);
  const [showViewEvent, setShowViewEvent] = useState(false);
  const [selectedEvent, setSelectedEvent] = useState<CaseEvent | null>(null);
  const [newEvent, setNewEvent] = useState<Omit<CaseEvent, '_id' | 'caseId' | 'createdAt' |
'updatedAt'>>({
    title: '',
    type: 'Deadline',
    date: '',
    time: '',
    description: '',
  });

  // Documents
  const [documents, setDocuments] = useState<CaseDocument[]>([]);
  const [docsLoading, setDocsLoading] = useState(false);
  const [docsError, setDocsError] = useState('');
  const [showUploadModal, setShowUploadModal] = useState(false);
  const [newDoc, setNewDoc] = useState<NewDocForm>({ name: '', file: undefined });
```

```
// Edit case modal
const [showEditCase, setShowEditCase] = useState(false);
const [editCaseData, setEditCaseData] = useState<CaseData | null>(null);

// Billing
const [invoices, setInvoices] = useState<Invoice[]>([]);
const [invoicesLoading, setInvoicesLoading] = useState(false);
const [invoicesError, setInvoicesError] = useState('');

const [showAddInvoiceModal, setShowAddInvoiceModal] = useState(false);
const [newInvoice, setNewInvoice] = useState({
  date: '',
  amount: '',
  notes: '',
});
const [newInvoiceFile, setNewInvoiceFile] = useState<File | null>(null);

const [showUploadProofModal, setShowUploadProofModal] = useState(false);
const [proofInvoiceId, setProofInvoiceId] = useState<string | null>(null);
const [proofFile, setProofFile] = useState<File | null>(null);
const [uploadingProof, setUploadingProof] = useState(false);

// ----------------------------
// Load case
// ----------------------------
useEffect(() => {
  if (!id) return;
  setLoading(true);
  setError('');
  getCaseById(id)
    .then((data) => setCaseData(data))
    .catch((err) => setError(err.message || 'Failed to load case'))
    .finally(() => setLoading(false));
}, [id]);

// ----------------------------
// Fetch staff list (only needed for MD/Exec actions)
// ----------------------------
useEffect(() => {
  if (!canManageCase && !canAssignTasks) return;

  const fetchStaff = async () => {
    try {
      setStaffLoading(true);
```

```
      setStaffError('');

      const token = getToken();
      const res = await fetch(`${API_URL}/users/staff`, {
        headers: token ? { Authorization: `Bearer ${token}` } : {},
      });

      if (res.status === 401) {
        localStorage.removeItem('token');
        localStorage.removeItem('user');
        navigate('/login');
        return;
      }

      if (!res.ok) {
        const text = await res.text();
        throw new Error(text || 'Failed to fetch staff users');
      }

      const data = (await res.json()) as StaffUser[];
      setStaffUsers(Array.isArray(data) ? data : []);
    } catch (e: any) {
      setStaffError(e?.message || 'Failed to fetch staff users');
      setStaffUsers([]);
    } finally {
      setStaffLoading(false);
    }
  };

  fetchStaff();
}, [navigate, canManageCase, canAssignTasks]);

// ----------------------------
// Tasks
// ----------------------------
const reloadTasks = async () => {
  if (!caseData?._id) return;
  setTasksLoading(true);
  setTasksError('');
  try {
    const data = await getTasksForCase(caseData._id);
    setTasks(data);
  } catch (err: any) {
    setTasksError(err.message || 'Failed to load tasks');
```

```
  } finally {
    setTasksLoading(false);
  }
};

useEffect(() => {
  reloadTasks();
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [caseData?._id]);

const handleAddTask = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!caseData?._id) return;

  if (!canAssignTasks) {
    setTasksError('You do not have permission to assign tasks.');
    return;
  }

  try {
    await addTaskToCase(caseData._id, { ...newTask, caseId: caseData._id });
    setShowAddTask(false);
    setNewTask({
      title: '',
      priority: 'Medium',
      status: 'Not Started',
      assignee: '',
      dueDate: '',
      description: '',
      requiresApproval: false,
    });
    reloadTasks();
  } catch (err: any) {
    setTasksError(err.message || 'Failed to add task');
  }
};

const handleEditTask = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!editTask?._id) return;

  if (!canAssignTasks) {
    setTasksError('You do not have permission to edit tasks here.');
    return;
```

```
  }

  try {
    await updateTask(editTask._id, editTask);
    setShowEditTask(false);
    setEditTask(null);
    reloadTasks();
  } catch (err: any) {
    setTasksError(err.message || 'Failed to update task');
  }
};

const handleDeleteTask = async (taskId: string) => {
  if (!window.confirm('Are you sure you want to delete this task?')) return;

  if (!canAssignTasks) {
    setTasksError('You do not have permission to delete tasks.');
    return;
  }

  try {
    await deleteTask(taskId);
    reloadTasks();
  } catch (err: any) {
    setTasksError(err.message || 'Failed to delete task');
  }
};

const handleStatusChange = async (task: TaskData, newStatus: string) => {
  if (isAssociate) return; // ✅ read-only in case workspace for associates
  try {
    await updateTask(task._id!, { status: newStatus });
    reloadTasks();
  } catch (err: any) {
    setTasksError(err.message || 'Failed to update status');
  }
};

// ----------------------------
// Events
// ----------------------------
const reloadEvents = async () => {
  if (!caseData?._id) return;
  try {
```

```
    const data = await getEventsForCase(caseData._id);
    setEvents(data);
  } catch {
    setEvents([]);
  }
};

useEffect(() => {
  reloadEvents();
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [caseData?._id, showAddEvent, showEditEvent]);

const handleAddEvent = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!caseData?._id) return;

  if (!canManageCalendar) return;

  await addEventToCase(caseData._id, { ...newEvent, caseId: caseData._id });
  setShowAddEvent(false);
  setNewEvent({ title: '', type: 'Deadline', date: '', time: '', description: '' });
  reloadEvents();
};

const handleEditEvent = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!selectedEvent?._id) return;

  if (!canManageCalendar) return;

  await updateEvent(selectedEvent._id, newEvent);
  setShowEditEvent(false);
  setSelectedEvent(null);
  setNewEvent({ title: '', type: 'Deadline', date: '', time: '', description: '' });
  reloadEvents();
};

const handleDeleteEvent = async (eventId: string) => {
  if (!window.confirm('Are you sure you want to delete this event?')) return;
  if (!canManageCalendar) return;

  await deleteEvent(eventId);
  setEvents((prev) => prev.filter((ev) => ev._id !== eventId));
  setShowViewEvent(false);
```

```
  setSelectedEvent(null);
};

const handleViewEvent = (event: CaseEvent) => {
  setSelectedEvent(event);
  setShowViewEvent(true);
};

const openEditEvent = (event: CaseEvent) => {
  if (!canManageCalendar) return;
  setSelectedEvent(event);
  setNewEvent({
    title: event.title,
    type: event.type,
    date: event.date,
    time: event.time,
    description: event.description || '',
  });
  setShowEditEvent(true);
};

// ----------------------------
// Documents
// ----------------------------
const reloadDocuments = async () => {
  if (!caseData?._id) return;
  setDocsLoading(true);
  setDocsError('');
  try {
    const data = await getDocumentsForCase(caseData._id);
    setDocuments(data);
  } catch (err: any) {
    setDocsError(err.message || 'Failed to fetch documents');
  } finally {
    setDocsLoading(false);
  }
};

useEffect(() => {
  reloadDocuments();
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [caseData?._id, showUploadModal]);

const handleUpload = async (e: React.FormEvent) => {
```

```
  e.preventDefault();
  if (!caseData?._id || !newDoc.name || !newDoc.file) return;

  if (!canManageDocuments) {
    setDocsError('You do not have permission to upload documents.');
    return;
  }

  setDocsError('');
  try {
    await addDocumentToCase(caseData._id, {
      name: newDoc.name,
      file: newDoc.file,
    });

    setShowUploadModal(false);
    setNewDoc({ name: '', file: undefined });
    reloadDocuments();
  } catch (err: any) {
    setDocsError(err.message || 'Failed to upload document');
  }
};

const handleDeleteDoc = async (docId: string) => {
  if (!window.confirm('Are you sure you want to delete this document?')) return;

  if (!canManageDocuments) {
    setDocsError('You do not have permission to delete documents.');
    return;
  }

  try {
    await deleteDocument(docId);
    setDocuments((prev) => prev.filter((d) => d._id !== docId));
  } catch (err: any) {
    setDocsError(err.message || 'Failed to delete document');
  }
};

// ----------------------------
// Edit case
// ----------------------------
const handleEditCase = async (e: React.FormEvent) => {
  e.preventDefault();
```

```
  if (!editCaseData?._id) return;

  if (!canManageCase) return;

  try {
    await updateCase(editCaseData._id, editCaseData);
    setShowEditCase(false);
    const updated = await getCaseById(editCaseData._id);
    setCaseData(updated);
  } catch (err: any) {
    alert(err.message || 'Failed to update case');
  }
};

// ----------------------------
// Billing
// ----------------------------
const reloadInvoices = async () => {
  if (!caseData?._id) return;
  setInvoicesLoading(true);
  setInvoicesError('');
  try {
    const data = await getInvoicesForCase(caseData._id);
    setInvoices(data);
  } catch (err: any) {
    setInvoicesError(err.message || 'Failed to fetch invoices');
    setInvoices([]);
  } finally {
    setInvoicesLoading(false);
  }
};

useEffect(() => {
  if (activeTab === 'billing' && canManageBilling) {
    reloadInvoices();
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [caseData?._id, activeTab]);

const handleAddInvoice = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!caseData?._id) return;
  if (!canManageBilling) return;
```

```
  const amountNum = Number(String(newInvoice.amount).replace(/[^\d.]/g, ''));
  if (!newInvoice.date || !Number.isFinite(amountNum) || amountNum <= 0) {
    setInvoicesError('Provide date and a valid amount.');
    return;
  }

  try {
    setInvoicesError('');
    const created = await addInvoiceToCase(caseData._id, {
      date: newInvoice.date,
      amount: amountNum,
      notes: newInvoice.notes,
    });

    if (created?._id && newInvoiceFile) {
      await uploadInvoiceFile(created._id, newInvoiceFile);
    }

    setShowAddInvoiceModal(false);
    setNewInvoice({ date: '', amount: '', notes: '' });
    setNewInvoiceFile(null);
    reloadInvoices();
  } catch (err: any) {
    setInvoicesError(err.message || 'Failed to create invoice');
  }
};

const openUploadProofModal = (invoiceId: string) => {
  if (!canManageBilling) return;
  setProofInvoiceId(invoiceId);
  setProofFile(null);
  setShowUploadProofModal(true);
};

const handleUploadProof = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!proofInvoiceId || !proofFile) return;
  if (!canManageBilling) return;

  try {
    setUploadingProof(true);
    await uploadProof(proofInvoiceId, proofFile);
    setShowUploadProofModal(false);
    setProofInvoiceId(null);
```

```
      setProofFile(null);
      reloadInvoices();
    } catch (err: any) {
      setInvoicesError(err.message || 'Failed to upload proof');
    } finally {
      setUploadingProof(false);
    }
  };

  const totalBilled = useMemo(() => parseBudgetToNumber(caseData?.budget),
[caseData?.budget]);
  const totalPaid = useMemo(
    () =>
      invoices.filter((i) => i.status === 'Paid').reduce((sum, i) => sum + (Number(i.amount) || 0), 0),
    [invoices]
  );
  const outstanding = useMemo(() => Math.max(0, totalBilled - totalPaid), [totalBilled, totalPaid]);

  const handleDeleteInvoice = async (invoiceId: string) => {
    if (!window.confirm('Are you sure you want to delete this invoice?')) return;
    if (!canManageBilling) return;

    try {
      setInvoicesError('');
      await deleteInvoice(invoiceId);
      reloadInvoices();
    } catch (err: any) {
      setInvoicesError(err.message || 'Failed to delete invoice');
    }
  };

  // ----------------------------
  // Audit log
  // ----------------------------
  const [auditLogs, setAuditLogs] = useState<AuditLogItem[]>([]);
  const [auditLoading, setAuditLoading] = useState(false);
  const [auditError, setAuditError] = useState('');

  const reloadAudit = async () => {
    if (!caseData?._id) return;
    setAuditLoading(true);
    setAuditError('');
    try {
      const data = await getAuditForCase(caseData._id);
```

```
    setAuditLogs(data);
  } catch (err: any) {
    setAuditError(err.message || 'Failed to load audit log');
    setAuditLogs([]);
  } finally {
    setAuditLoading(false);
  }
};

useEffect(() => {
  if (activeTab === 'audit') reloadAudit();
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [activeTab, caseData?._id]);

// Progress
const currentStage = caseData?.status || 'On Boarding';
const currentStageIndex = STAGE_ORDER.findIndex((s) => s === currentStage);
const progress = Math.round(((currentStageIndex + 1) / STAGE_ORDER.length) * 100);

const getStageStatus = (stage: string) => {
  if (STAGE_ORDER.indexOf(stage) < currentStageIndex) return 'Completed';
  if (stage === currentStage) return 'In Progress';
  return 'Pending';
};

const getStageColor = (status: string) => {
  if (status === 'Completed') return 'text-green-700 font-medium';
  if (status === 'In Progress') return 'text-blue-700 font-medium';
  return 'text-gray-500';
};

if (loading) return <div className="text-center py-12 text-gray-500">Loading case...</div>;
if (error) return <div className="text-center py-12 text-red-600">{error}</div>;
if (!caseData) return <div className="text-center py-12 text-gray-500">Case not found.</div>;

return (
  <div>
    {/* Header */}
    <div className="mb-6">
      <button
        onClick={() => navigate('/cases')}
        className="inline-flex items-center text-sm text-gray-600 hover:text-gray-900 mb-4"
      >
        <ArrowLeft className="w-4 h-4 mr-1" />
```

```
        Back to Cases
      </button>

      <div className="flex items-start justify-between mb-2">
        <div>
          <div className="flex items-center gap-3 mb-2">
            <h1 className="text-3xl font-bold text-gray-900">{caseData.parties}</h1>
            <span className="px-2 py-0.5 text-xs rounded bg-red-100 text-red-700
font-semibold">
              {caseData.priority}
            </span>
            <span className="px-2 py-0.5 text-xs rounded bg-green-100 text-green-700
font-semibold">
              {caseData.status}
            </span>
          </div>
          <p className="text-base text-gray-500">
            {caseData.caseNo} &bull; {caseData.caseType}
          </p>
        </div>

        {/* ✅ Edit Case hidden for associates */}
        {canManageCase && (
          <button
            className="px-5 py-2 border border-gray-300 rounded-lg text-base text-gray-700
hover:bg-gray-50 font-medium"
            onClick={() => {
              setEditCaseData(caseData);
              setShowEditCase(true);
            }}
          >
            Edit Case
          </button>
        )}
      </div>
    </div>

    {/* Tabs */}
    <div className="border-b border-gray-200 mb-6">
      <nav className="flex space-x-6 overflow-x-auto">
        {[
          { id: 'overview', label: 'Overview', icon: FileText },
          { id: 'tasks', label: 'Tasks', icon: CheckSquare },
          { id: 'calendar', label: 'Calendar', icon: CalendarIcon },
```

```
        { id: 'documents', label: 'Documents', icon: Upload },
        // ✅ Billing hidden for associates
        ...(canManageBilling ? [{ id: 'billing', label: 'Billing', icon: DollarSign }] : []),

        // ✅ NEW: Client Reports (MD/Exec only)
        ...(canManageCase ? [{ id: 'reports', label: 'Client Reports', icon: FileText }] : []),

        { id: 'audit', label: 'Audit Log', icon: Clock },
      ].map((tab) => {
        const Icon = tab.icon;
        return (
          <button
            key={tab.id}
            onClick={() => setActiveTab(tab.id as any)}
            className={`
              flex items-center px-1 py-3 border-b-2 font-medium text-sm whitespace-nowrap
transition-colors
              ${
                activeTab === tab.id
                  ? 'border-gray-900 text-gray-900'
                  : 'border-transparent text-gray-500 hover:text-gray-700'
              }
            `}
          >
            <Icon className="w-4 h-4 mr-2" />
            {tab.label}
          </button>
        );
      })}
    </nav>
  </div>

  {/* Overview */}
  {activeTab === 'overview' && (
    <div className="space-y-6">
      <div className="bg-white border border-gray-200 rounded-lg p-6">
        <h2 className="font-semibold text-gray-900 mb-4">Case Progress</h2>
        <div className="flex items-center justify-between text-sm text-gray-600 mb-2">
          <span>Overall Progress</span>
          <span className="font-semibold text-gray-800">{progress}%</span>
        </div>
        <div className="h-3 bg-gray-100 rounded-full overflow-hidden mb-6">
          <div className="h-full bg-gray-800 transition-all" style={{ width: `${progress}%` }} />
        </div>
```

```
      <div>
        {STAGE_ORDER.map((stage, idx) => {
          const s = getStageStatus(stage);
          return (
            <div key={stage} className="flex items-center justify-between py-2">
              <div className="flex items-center gap-3">
                <span className="text-gray-500 font-mono">{idx + 1}</span>
                <span className="text-gray-900">{stage}</span>
              </div>
              <span className={getStageColor(s)}>{s}</span>
            </div>
          );
        })}
      </div>
    </div>

    <div className="bg-white border border-gray-200 rounded-lg p-6">
      <h2 className="font-semibold text-gray-900 mb-3">Case Description</h2>
      <p className="text-sm text-gray-600 whitespace-pre-line">{caseData.description}</p>
    </div>
  </div>
)}

{/* Tasks */}
{activeTab === 'tasks' && (
  <div className="bg-white border border-gray-200 rounded-lg">
    <div className="px-5 py-4 border-b border-gray-200 flex items-center justify-between">
      <h2 className="font-semibold text-gray-900">Case Tasks</h2>

      {/* ✅ Add Task hidden for associates */}
      {canAssignTasks && (
        <button
          className="px-3 py-1 text-sm bg-gray-800 text-white rounded hover:bg-gray-700"
          onClick={() => setShowAddTask(true)}
        >
          + Add Task
        </button>
      )}
    </div>

    {tasksError && <div className="px-5 py-3 text-red-600">{tasksError}</div>}

    {tasksLoading ? (
      <div className="px-5 py-8 text-gray-500">Loading tasks...</div>
```

```
        ) : tasks.length === 0 ? (
          <div className="px-5 py-8 text-gray-500">No tasks yet for this case.</div>
        ) : (
          <div className="overflow-x-auto">
            <table className="w-full">
              <thead className="bg-gray-50 border-b">
                <tr>
                  <th className="px-4 py-2 text-left text-xs font-medium text-gray-600
uppercase">No.</th>
                  <th className="px-4 py-2 text-left text-xs font-medium text-gray-600
uppercase">Title</th>
                  <th className="px-4 py-2 text-left text-xs font-medium text-gray-600
uppercase">Priority</th>
                  <th className="px-4 py-2 text-left text-xs font-medium text-gray-600
uppercase">Status</th>
                  <th className="px-4 py-2 text-left text-xs font-medium text-gray-600
uppercase">Assignee</th>
                  <th className="px-4 py-2 text-left text-xs font-medium text-gray-600
uppercase">Due Date</th>
                  <th className="px-4 py-2 text-left text-xs font-medium text-gray-600
uppercase">Actions</th>
                </tr>
              </thead>

              <tbody className="divide-y divide-gray-200">
                {tasks.map((task, index) => (
                  <tr key={task._id}>
                    <td className="px-4 py-3 text-gray-500">{index + 1}</td>
                    <td className="px-4 py-3">{task.title}</td>
                    <td className="px-4 py-3">
                      <span
                        className={
                          task.priority === 'High'
                            ? 'text-red-700 font-semibold'
                            : task.priority === 'Medium'
                              ? 'text-yellow-700 font-semibold'
                              : 'text-green-700 font-semibold'
                        }
                      >
                        {task.priority}
                      </span>
                    </td>

                    {/* ✅ Associates read-only status */}
```

```
<td className="px-4 py-3">
  {isAssociate ? (
    <span className="text-sm text-gray-700">{task.status}</span>
  ) : (
    <select
      value={task.status}
      onChange={(e) => handleStatusChange(task, e.target.value)}
      className="border border-gray-300 rounded px-2 py-1 text-xs"
    >
      <option value="Not Started">Not Started</option>
      <option value="In Progress">In Progress</option>
      <option value="Completed">Completed</option>
    </select>
  )}
</td>

<td className="px-4 py-3">{task.assignee}</td>
<td className="px-4 py-3">{task.dueDate}</td>

<td className="px-4 py-3">
  <div className="flex items-center gap-2">
    {/* ✅ Everyone can open task detail */}
    <button
      onClick={() => navigate(`/tasks/${task._id}`)}
      className="text-gray-700 hover:text-gray-900"
      title="Open task"
    >
      <Eye className="w-4 h-4" />
    </button>

    {/* ✅ Only MD/Exec can edit/delete */}
    {canAssignTasks && (
      <>
        <button
          onClick={() => {
            setEditTask(task);
            setShowEditTask(true);
          }}
          className="text-blue-600 hover:text-blue-900"
          title="Edit"
        >
          <Edit className="w-4 h-4" />
        </button>
```

```
                <button
                  onClick={() => handleDeleteTask(task._id!)}
                  className="text-red-600 hover:text-red-900"
                  title="Delete"
                >
                  <Trash2 className="w-4 h-4" />
                </button>

                {task.status !== 'Completed' && (
                  <button
                    onClick={() => handleStatusChange(task, 'Completed')}
                    className="text-green-600 hover:text-green-900"
                    title="Mark as Completed"
                  >
                    <Check className="w-4 h-4" />
                  </button>
                )}
              </>
            )}
          </div>
        </td>
      </tr>
    ))}
  </tbody>
</table>

{isAssociate && (
  <div className="px-5 py-4 text-xs text-gray-500 border-t">
    Note: Task progress updates should be done from the Task page.
  </div>
)}
</div>
)}
</div>
)}

{/* ✅ Client Reports (MD/Exec only) */}
{activeTab === 'reports' && canManageCase && (
  <CaseClientReportsTab caseData={caseData} canManage={canManageCase} />
)}

{/* ✅ Add Task Modal (only for MD/Exec) */}
{showAddTask && canAssignTasks && (
```

```
      <div className="fixed inset-0 bg-gray-900 bg-opacity-50 flex items-center justify-center
z-50 p-4">
        <div className="bg-white rounded-lg max-w-lg w-full flex flex-col" style={{ maxHeight:
'90vh' }}>
          <div className="p-6 border-b">
            <h3 className="text-lg font-semibold text-gray-900">Add Task</h3>
          </div>

          <form onSubmit={handleAddTask} className="flex-1 overflow-y-auto space-y-4 p-6">
            <div>
              <label className="block text-sm font-medium text-gray-700 mb-1">Title</label>
              <input
                type="text"
                value={newTask.title}
                onChange={(e) => setNewTask((t) => ({ ...t, title: e.target.value }))}
                required
                className="w-full px-3 py-2 border border-gray-300 rounded"
              />
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700 mb-1">Priority</label>
              <select
                value={newTask.priority}
                onChange={(e) => setNewTask((t) => ({ ...t, priority: e.target.value as any }))}
                className="w-full px-3 py-2 border border-gray-300 rounded"
              >
                <option value="High">High</option>
                <option value="Medium">Medium</option>
                <option value="Low">Low</option>
              </select>
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700 mb-1">Initial
Status</label>
              <select
                value={newTask.status}
                onChange={(e) => setNewTask((t) => ({ ...t, status: e.target.value as any }))}
                className="w-full px-3 py-2 border border-gray-300 rounded"
              >
                <option value="Not Started">Not Started</option>
                <option value="In Progress">In Progress</option>
                <option value="Completed">Completed</option>
```

```
          </select>
        </div>

        <div>
          <label className="block text-sm font-medium text-gray-700 mb-1">Assignee</label>
          <select
            value={newTask.assignee}
            onChange={(e) => setNewTask((t) => ({ ...t, assignee: e.target.value }))}
            className="w-full px-3 py-2 border border-gray-300 rounded"
            disabled={staffLoading}
            required
          >
            <option value="">{staffLoading ? 'Loading...' : 'Select assignee'}</option>
            {staffUsers.map((u) => (
              <option key={u._id} value={u.name}>
                {u.name}
              </option>
            ))}
          </select>

          {staffError && <p className="text-xs text-red-600 mt-2">{staffError}</p>}
        </div>

        <div>
          <label className="block text-sm font-medium text-gray-700 mb-1">Due Date</label>
          <input
            type="date"
            value={newTask.dueDate}
            onChange={(e) => setNewTask((t) => ({ ...t, dueDate: e.target.value }))}
            className="w-full px-3 py-2 border border-gray-300 rounded"
          />
        </div>

        <div>
          <label className="block text-sm font-medium text-gray-700
mb-1">Description</label>
          <textarea
            rows={2}
            value={newTask.description || ''}
            onChange={(e) => setNewTask((t) => ({ ...t, description: e.target.value }))}
            className="w-full px-3 py-2 border border-gray-300 rounded"
          />
        </div>
```

```
            {userRole === 'managing_director' && (
              <div className="flex items-center justify-between">
                <span className="text-sm text-gray-700">Requires Approval</span>
                <input
                  type="checkbox"
                  checked={Boolean(newTask.requiresApproval)}
                  onChange={(e) => setNewTask((t) => ({ ...t, requiresApproval: e.target.checked }))}
                  className="h-4 w-4"
                />
              </div>
            )}

            <div className="flex gap-3 pt-2">
              <button
                type="button"
                onClick={() => setShowAddTask(false)}
                className="flex-1 px-4 py-2 border border-gray-300 rounded text-gray-700
hover:bg-gray-50"
              >
                Cancel
              </button>

              <button type="submit" className="flex-1 px-4 py-2 bg-gray-800 text-white rounded
hover:bg-gray-700">
                Add Task
              </button>
            </div>
          </form>
        </div>
      </div>
    )}

    {/* ✅ Edit Task Modal only for MD/Exec */}
    {showEditTask && editTask && canAssignTasks && (
      <div className="fixed inset-0 bg-gray-900 bg-opacity-50 flex items-center justify-center
z-50 p-4">
        <div className="bg-white rounded-lg max-w-lg w-full flex flex-col" style={{ maxHeight:
'90vh' }}>
          <div className="p-6 border-b">
            <h3 className="text-lg font-semibold text-gray-900">Edit Task</h3>
          </div>

          <form id="edit-task-form" onSubmit={handleEditTask} className="flex-1
overflow-y-auto space-y-4 p-6">
```

```
<div>
  <label className="block text-sm font-medium text-gray-700 mb-1">Title</label>
  <input
    type="text"
    value={editTask.title}
    onChange={(e) => setEditTask((t) => (t ? { ...t, title: e.target.value } : t))}
    className="w-full px-3 py-2 border border-gray-300 rounded"
  />
</div>

<div>
  <label className="block text-sm font-medium text-gray-700 mb-1">Priority</label>
  <select
    value={editTask.priority}
    onChange={(e) => setEditTask((t) => (t ? { ...t, priority: e.target.value as any } : t))}
    className="w-full px-3 py-2 border border-gray-300 rounded"
  >
    <option value="High">High</option>
    <option value="Medium">Medium</option>
    <option value="Low">Low</option>
  </select>
</div>

<div>
  <label className="block text-sm font-medium text-gray-700 mb-1">Status</label>
  <select
    value={editTask.status}
    onChange={(e) => setEditTask((t) => (t ? { ...t, status: e.target.value as any } : t))}
    className="w-full px-3 py-2 border border-gray-300 rounded"
  >
    <option value="Not Started">Not Started</option>
    <option value="In Progress">In Progress</option>
    <option value="Completed">Completed</option>
  </select>
</div>

<div>
  <label className="block text-sm font-medium text-gray-700 mb-1">Assignee</label>
  <select
    value={editTask.assignee || ''}
    onChange={(e) => setEditTask((t) => (t ? { ...t, assignee: e.target.value } : t))}
    className="w-full px-3 py-2 border border-gray-300 rounded"
    disabled={staffLoading}
    required
```

```
            >
              <option value="">{staffLoading ? 'Loading...' : 'Select assignee'}</option>
              {staffUsers.map((u) => (
                <option key={u._id} value={u.name}>
                  {u.name}
                </option>
              ))}
            </select>

            {staffError && <p className="text-xs text-red-600 mt-2">{staffError}</p>}
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Due Date</label>
            <input
              type="date"
              value={editTask.dueDate}
              onChange={(e) => setEditTask((t) => (t ? { ...t, dueDate: e.target.value } : t))}
              className="w-full px-3 py-2 border border-gray-300 rounded"
            />
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700
mb-1">Description</label>
            <textarea
              rows={2}
              value={editTask.description || ''}
              onChange={(e) => setEditTask((t) => (t ? { ...t, description: e.target.value } : t))}
              className="w-full px-3 py-2 border border-gray-300 rounded"
            />
          </div>

          <div className="border-t pt-4 space-y-3">
            {userRole === 'managing_director' && (
              <>
                <div className="flex items-center justify-between">
                  <span className="text-sm text-gray-700">Requires Approval</span>
                  <input
                    type="checkbox"
                    checked={Boolean((editTask as any).requiresApproval)}
                    onChange={(e) =>
                      setEditTask((t) => (t ? ({ ...t, requiresApproval: e.target.checked } as any) : t))
                    }
```

```
                  className="h-4 w-4"
                />
              </div>

              <div className="flex flex-wrap gap-2">
                <button
                  type="button"
                  onClick={handleSubmitForApproval}
                  disabled={approvalLoading}
                  className="px-3 py-2 text-sm bg-gray-100 rounded hover:bg-gray-200
disabled:opacity-60"
                >
                  Submit for approval
                </button>

                <input
                  value={approvalComment}
                  onChange={(e) => setApprovalComment(e.target.value)}
                  placeholder="Approval comment (optional)"
                  className="flex-1 min-w-[200px] px-3 py-2 border border-gray-300 rounded
text-sm"
                />

                <button
                  type="button"
                  onClick={handleApprove}
                  disabled={approvalLoading}
                  className="px-3 py-2 text-sm bg-green-600 text-white rounded
hover:bg-green-700 disabled:opacity-60"
                >
                  Approve
                </button>

                <button
                  type="button"
                  onClick={handleReject}
                  disabled={approvalLoading}
                  className="px-3 py-2 text-sm bg-red-600 text-white rounded hover:bg-red-700
disabled:opacity-60"
                >
                  Reject
                </button>
              </div>
            </>
```

```
            )}
          </div>
        </form>

        <div className="flex gap-3 p-6 border-t">
          <button
            type="button"
            onClick={() => setShowEditTask(false)}
            className="flex-1 px-4 py-2 border border-gray-300 rounded text-gray-700
hover:bg-gray-50"
          >
            Cancel
          </button>

          <button
            type="submit"
            form="edit-task-form"
            className="flex-1 px-4 py-2 bg-gray-800 text-white rounded hover:bg-gray-700"
          >
            Save Changes
          </button>
        </div>
      </div>
    </div>
  )}

        {/* Calendar */}
  {activeTab === 'calendar' && (
    <div className="bg-white border border-gray-200 rounded-lg overflow-hidden">
      <div className="flex items-center justify-between px-5 py-4 border-b border-gray-200">
        <div>
          <h2 className="font-semibold text-gray-900">Case Events & Deadlines</h2>
          <p className="text-sm text-gray-500 mt-1">Deadlines, hearings, meetings and
milestones for this case.</p>
        </div>

        {canManageCalendar && (
          <button
            className="px-4 py-2 bg-gray-900 text-white rounded hover:bg-gray-800"
            onClick={() => {
              setNewEvent({ title: '', type: 'Deadline', date: '', time: '', description: '' });
              setSelectedEvent(null);
              setShowAddEvent(true);
            }}
```

```
          >
            + Add Event
          </button>
        )}
      </div>

      <div>
        {events.length === 0 ? (
          <div className="text-center py-10 text-gray-500">No events yet for this case.</div>
        ) : (
          <div className="divide-y divide-gray-100">
            {events.map((event) => (
              <div
                key={event._id}
                className="flex flex-col md:flex-row md:items-center justify-between gap-4 px-5
py-6"
              >
                <div className="min-w-0">
                  <span className="inline-block px-3 py-1 bg-gray-100 text-gray-700 text-xs
rounded mb-2">
                    {event.type}
                  </span>
                  <div className="font-medium text-gray-900 text-lg truncate">{event.title}</div>
                  <div className="flex items-center text-sm text-gray-600 mt-1">
                    <CalendarIcon className="w-4 h-4 mr-1" />
                    {event.date} at {event.time}
                  </div>
                  {event.description ? (
                    <div className="text-sm text-gray-500 mt-2 whitespace-pre-line">
                      {event.description}
                    </div>
                  ) : null}
                </div>

                <div className="flex gap-2 mt-2 md:mt-0 shrink-0">
                  <button
                    onClick={() => handleViewEvent(event)}
                    className="p-2 text-gray-600 hover:text-blue-700"
                    title="View"
                  >
                    <Eye className="w-4 h-4" />
                  </button>

                  {canManageCalendar && (
```

```
                <>
                  <button
                    onClick={() => openEditEvent(event)}
                    className="p-2 text-gray-600 hover:text-blue-700"
                    title="Edit"
                  >
                    <Edit className="w-4 h-4" />
                  </button>

                  <button
                    onClick={() => handleDeleteEvent(event._id!)}
                    className="p-2 text-gray-600 hover:text-red-700"
                    title="Delete"
                  >
                    <Trash2 className="w-4 h-4" />
                  </button>
                <//>
              )}
            </div>
          </div>
        ))}
      </div>
    )}
  </div>

  {/* Add Event Modal */}
  {showAddEvent && canManageCalendar && (
    <div className="fixed inset-0 bg-gray-900 bg-opacity-50 flex items-center justify-center
z-50 p-4">
      <div className="bg-white rounded-lg w-full max-w-lg flex flex-col" style={{ maxHeight:
'90vh' }}>
        <div className="p-6 border-b flex items-center justify-between">
          <h3 className="text-lg font-semibold text-gray-900">Add Event</h3>
          <button
            type="button"
            onClick={() => setShowAddEvent(false)}
            className="px-3 py-2 border border-gray-300 rounded text-sm text-gray-700
hover:bg-gray-50"
          >
            Close
          </button>
        </div>

        <form onSubmit={handleAddEvent} className="p-6 space-y-4 overflow-y-auto">
```

```
            <div>
              <label className="block text-sm font-medium text-gray-700 mb-1">Title *</label>
              <input
                value={newEvent.title}
                onChange={(e) => setNewEvent((p) => ({ ...p, title: e.target.value }))}
                required
                className="w-full px-3 py-2 border border-gray-300 rounded"
                placeholder="e.g. Hearing / Filing Deadline"
              />
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700 mb-1">Type</label>
              <select
                value={newEvent.type}
                onChange={(e) => setNewEvent((p) => ({ ...p, type: e.target.value }))}
                className="w-full px-3 py-2 border border-gray-300 rounded"
              >
                {eventTypes.map((t) => (
                  <option key={t} value={t}>
                    {t}
                  </option>
                ))}
              </select>
            </div>

            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
              <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">Date
*</label>
                <input
                  type="date"
                  value={newEvent.date}
                  onChange={(e) => setNewEvent((p) => ({ ...p, date: e.target.value }))}
                  required
                  className="w-full px-3 py-2 border border-gray-300 rounded"
                />
              </div>

              <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">Time
*</label>
                <input
                  type="time"
```

```
                value={newEvent.time}
                onChange={(e) => setNewEvent((p) => ({ ...p, time: e.target.value }))}
                required
                className="w-full px-3 py-2 border border-gray-300 rounded"
              />
            </div>
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700
mb-1">Description</label>
            <textarea
              rows={3}
              value={newEvent.description || ''}
              onChange={(e) => setNewEvent((p) => ({ ...p, description: e.target.value }))}
              className="w-full px-3 py-2 border border-gray-300 rounded"
              placeholder="Optional notes for the team..."
            />
          </div>

          <div className="flex gap-3 pt-2">
            <button
              type="button"
              onClick={() => setShowAddEvent(false)}
              className="flex-1 px-4 py-2 border border-gray-300 rounded text-gray-700
hover:bg-gray-50"
            >
              Cancel
            </button>
            <button
              type="submit"
              className="flex-1 px-4 py-2 bg-gray-900 text-white rounded hover:bg-gray-800"
            >
              Add Event
            </button>
          </div>
        </form>
      </div>
    </div>
  )}

  {/* Edit Event Modal */}
  {showEditEvent && selectedEvent && canManageCalendar && (
```

```
        <div className="fixed inset-0 bg-gray-900 bg-opacity-50 flex items-center justify-center
z-50 p-4">
          <div className="bg-white rounded-lg w-full max-w-lg flex flex-col" style={{ maxHeight:
'90vh' }}>
            <div className="p-6 border-b flex items-center justify-between">
              <h3 className="text-lg font-semibold text-gray-900">Edit Event</h3>
              <button
                type="button"
                onClick={() => {
                  setShowEditEvent(false);
                  setSelectedEvent(null);
                }}
                className="px-3 py-2 border border-gray-300 rounded text-sm text-gray-700
hover:bg-gray-50"
              >
                Close
              </button>
            </div>

            <form onSubmit={handleEditEvent} className="p-6 space-y-4 overflow-y-auto">
              <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">Title *</label>
                <input
                  value={newEvent.title}
                  onChange={(e) => setNewEvent((p) => ({ ...p, title: e.target.value }))}
                  required
                  className="w-full px-3 py-2 border border-gray-300 rounded"
                />
              </div>

              <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">Type</label>
                <select
                  value={newEvent.type}
                  onChange={(e) => setNewEvent((p) => ({ ...p, type: e.target.value }))}
                  className="w-full px-3 py-2 border border-gray-300 rounded"
                >
                  {eventTypes.map((t) => (
                    <option key={t} value={t}>
                      {t}
                    </option>
                  ))}
                </select>
              </div>
```

```
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
              <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">Date
*</label>
                <input
                  type="date"
                  value={newEvent.date}
                  onChange={(e) => setNewEvent((p) => ({ ...p, date: e.target.value }))}
                  required
                  className="w-full px-3 py-2 border border-gray-300 rounded"
                />
              </div>

              <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">Time
*</label>
                <input
                  type="time"
                  value={newEvent.time}
                  onChange={(e) => setNewEvent((p) => ({ ...p, time: e.target.value }))}
                  required
                  className="w-full px-3 py-2 border border-gray-300 rounded"
                />
              </div>
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700
mb-1">Description</label>
              <textarea
                rows={3}
                value={newEvent.description || ''}
                onChange={(e) => setNewEvent((p) => ({ ...p, description: e.target.value }))}
                className="w-full px-3 py-2 border border-gray-300 rounded"
              />
            </div>

            <div className="flex gap-3 pt-2">
              <button
                type="button"
                onClick={() => {
                  setShowEditEvent(false);
                  setSelectedEvent(null);
```

```
                }}
                className="flex-1 px-4 py-2 border border-gray-300 rounded text-gray-700
hover:bg-gray-50"
              >
                Cancel
              </button>
              <button
                type="submit"
                className="flex-1 px-4 py-2 bg-gray-900 text-white rounded hover:bg-gray-800"
              >
                Save Changes
              </button>
            </div>
          </form>
        </div>
      </div>
    )}

    {/* View Event Modal */}
    {showViewEvent && selectedEvent && (
      <div className="fixed inset-0 bg-gray-900 bg-opacity-50 flex items-center justify-center
z-50 p-4">
        <div className="bg-white rounded-lg w-full max-w-lg">
          <div className="p-6 border-b flex items-center justify-between">
            <h3 className="text-lg font-semibold text-gray-900">Event Details</h3>
            <button
              type="button"
              onClick={() => {
                setShowViewEvent(false);
                setSelectedEvent(null);
              }}
              className="px-3 py-2 border border-gray-300 rounded text-sm text-gray-700
hover:bg-gray-50"
            >
              Close
            </button>
          </div>

          <div className="p-6 space-y-3">
            <div>
              <div className="text-xs text-gray-500">Type</div>
              <div className="text-sm text-gray-900 font-medium">{selectedEvent.type}</div>
            </div>
```

```
            <div>
              <div className="text-xs text-gray-500">Title</div>
              <div className="text-sm text-gray-900 font-medium">{selectedEvent.title}</div>
            </div>

            <div>
              <div className="text-xs text-gray-500">When</div>
              <div className="text-sm text-gray-900 font-medium">
                {selectedEvent.date} at {selectedEvent.time}
              </div>
            </div>

            {selectedEvent.description ? (
              <div>
                <div className="text-xs text-gray-500">Description</div>
                <div className="text-sm text-gray-700
whitespace-pre-line">{selectedEvent.description}</div>
              </div>
            ) : null}

            <div className="pt-4 flex gap-2 justify-end">
              {canManageCalendar ? (
                <>
                  <button
                    type="button"
                    onClick={() => {
                      setShowViewEvent(false);
                      openEditEvent(selectedEvent);
                    }}
                    className="px-4 py-2 border border-gray-300 rounded text-sm text-gray-700
hover:bg-gray-50"
                  >
                    Edit
                  </button>
                  <button
                    type="button"
                    onClick={() => handleDeleteEvent(selectedEvent._id!)}
                    className="px-4 py-2 border border-red-200 rounded text-sm text-red-700
hover:bg-red-50"
                  >
                    Delete
                  </button>
                </>
              ) : null}
```

```
                <button
                  type="button"
                  onClick={() => {
                    setShowViewEvent(false);
                    setSelectedEvent(null);
                  }}
                  className="px-4 py-2 bg-gray-900 text-white rounded text-sm
hover:bg-gray-800"
                >
                  Done
                </button>
              </div>
            </div>
          </div>
        </div>
      )}
    </div>
  )}

  {/* Documents */}
  {activeTab === 'documents' && (
    <div className="bg-white border border-gray-200 rounded-lg overflow-hidden">
      <div className="flex items-center justify-between px-6 py-5 border-b border-gray-200">
        <div>
          <h2 className="text-lg font-semibold text-gray-900">
            Case Documents <span className="text-gray-500
font-medium">({documents.length})</span>
          </h2>
          <p className="text-sm text-gray-500">Files linked to this case</p>
        </div>

        {canManageDocuments && (
          <button
            className="flex items-center gap-2 px-4 py-2 bg-gray-900 text-white rounded-lg
hover:bg-gray-800"
            onClick={() => setShowUploadModal(true)}
          >
            <Upload className="w-4 h-4" />
            Upload
          </button>
        )}
      </div>

      {docsError && (
```

```
          <div className="px-6 py-3 text-sm text-red-700 bg-red-50 border-b border-red-100">
            {docsError}
          </div>
        )}

        {docsLoading ? (
          <div className="px-6 py-10 text-gray-500">Loading documents...</div>
        ) : documents.length === 0 ? (
          <div className="px-6 py-10 text-gray-500">No documents yet for this case.</div>
        ) : (
          <div className="divide-y divide-gray-100">
            {documents.map((doc, index) => {
              const fileUrl = BACKEND_URL + doc.url;

              return (
                <div key={doc._id} className="px-6 py-5 flex items-center justify-between gap-4">
                  <div className="flex items-start gap-4 min-w-0">
                    <div className="w-7 text-sm text-gray-400 font-medium pt-0.5">{index +
1}.</div>
                    <FileText className="w-6 h-6 text-gray-600 mt-0.5 shrink-0" />

                    <div className="min-w-0">
                      <div className="font-semibold text-gray-900 truncate">{doc.name}</div>
                      <div className="text-sm text-gray-500 mt-1">
                        Uploaded by <span className="font-medium">{doc.uploadedBy}</span> on{'
'}
                        <span className="font-medium">{doc.uploadedDate}</span> • {doc.size}
                      </div>
                    </div>
                  </div>

                  <div className="flex items-center gap-2 shrink-0">
                    <a
                      href={fileUrl}
                      target="_blank"
                      rel="noopener noreferrer"
                      className="inline-flex items-center gap-2 px-3 py-2 text-sm border
border-gray-300 rounded-lg text-gray-700 hover:bg-gray-50"
                    >
                      <Eye className="w-4 h-4" />
                      <span className="hidden sm:inline">View</span>
                    </a>

                    <a
```

```
                    href={fileUrl}
                    download
                    className="inline-flex items-center gap-2 px-3 py-2 text-sm border
border-gray-300 rounded-lg text-gray-700 hover:bg-gray-50"
                  >
                    <ArrowDownIcon />
                    <span className="hidden sm:inline">Download</span>
                  </a>

                  {canManageDocuments && (
                    <button
                      onClick={() => handleDeleteDoc(doc._id!)}
                      className="inline-flex items-center gap-2 px-3 py-2 text-sm border
border-red-200 rounded-lg text-red-700 hover:bg-red-50"
                    >
                      <Trash2 className="w-4 h-4" />
                      <span className="hidden sm:inline">Delete</span>
                    </button>
                  )}
                </div>
              </div>
            );
          })}
        </div>
      )}

      {showUploadModal && canManageDocuments && (
        <div className="fixed inset-0 bg-gray-900 bg-opacity-50 flex items-center justify-center
z-50 p-4">
          <div className="bg-white rounded-lg max-w-md w-full p-6">
            <h3 className="text-lg font-semibold text-gray-900 mb-4">Upload Document</h3>

            <form onSubmit={handleUpload} className="space-y-4">
              <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">Document
Name *</label>
                <input
                  type="text"
                  value={newDoc.name}
                  onChange={(e) => setNewDoc((d) => ({ ...d, name: e.target.value }))}
                  required
                  className="w-full px-3 py-2 border border-gray-300 rounded"
                />
              </div>
```

```
              <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">File *</label>
                <input
                  type="file"
                  required
                  onChange={(e) => {
                    const file = (e.target.files && e.target.files[0]) || undefined;
                    setNewDoc((d) => ({ ...d, file }));
                  }}
                  className="w-full"
                />
              </div>

              <div className="flex gap-3 mt-6">
                <button
                  type="button"
                  onClick={() => setShowUploadModal(false)}
                  className="flex-1 px-4 py-2 border border-gray-300 rounded text-gray-700
hover:bg-gray-50"
                >
                  Cancel
                </button>
                <button
                  type="submit"
                  className="flex-1 px-4 py-2 bg-gray-800 text-white rounded hover:bg-gray-700"
                >
                  Upload
                </button>
              </div>
            </form>
          </div>
        </div>
      )}
    </div>
  )}

  {/* Billing */}
  {activeTab === 'billing' && (
    <div className="space-y-6">
      {/* Summary */}
      <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
        <div className="bg-white border border-gray-200 rounded-xl p-6">
          <span className="text-gray-500 text-sm mb-2 block">Total Billed (Budget)</span>
```

```
            <span className="text-3xl font-bold text-gray-900">{formatRwf(totalBilled)}</span>
          </div>
          <div className="bg-white border border-gray-200 rounded-xl p-6">
            <span className="text-gray-500 text-sm mb-2 block">Total Paid</span>
            <span className="text-3xl font-bold text-green-600">{formatRwf(totalPaid)}</span>
          </div>
          <div className="bg-white border border-gray-200 rounded-xl p-6">
            <span className="text-gray-500 text-sm mb-2 block">Outstanding</span>
            <span className="text-3xl font-bold
text-yellow-600">{formatRwf(outstanding)}</span>
          </div>
        </div>

        {/* Invoice list */}
        <div className="bg-white border border-gray-100 rounded-2xl overflow-hidden">
          <div className="px-6 py-5 border-b flex items-center justify-between">
            <div>
              <h3 className="text-lg font-semibold text-gray-900">
                Invoices <span className="text-gray-500 font-medium">({invoices.length})</span>
              </h3>
              <p className="text-sm text-gray-500">Invoices linked to this case only</p>
            </div>

            <button
              onClick={() => setShowAddInvoiceModal(true)}
              className="inline-flex items-center gap-2 px-4 py-2 bg-gray-900 text-white
rounded-lg hover:bg-gray-800"
            >
              <Plus className="w-4 h-4" />
              New Invoice
            </button>
          </div>

          {invoicesError && (
            <div className="px-6 py-3 text-sm text-red-700 bg-red-50
border-b">{invoicesError}</div>
          )}

          {invoicesLoading ? (
            <div className="px-6 py-10 text-gray-500">Loading invoices...</div>
          ) : invoices.length === 0 ? (
            <div className="px-6 py-10 text-gray-500">No invoices yet for this case.</div>
          ) : (
            <div className="divide-y">
```

```
            {invoices.map((inv, index) => (
              <div key={inv._id} className="px-6 py-5 flex items-center justify-between gap-4">
                {/* Left */}
                <div className="flex items-start gap-4 min-w-0">
                  {/* numbering */}
                  <div className="w-7 text-sm text-gray-400 font-medium pt-0.5">{index +
1}.</div>

                  <Receipt className="w-5 h-5 text-gray-600 mt-1 shrink-0" />

                  <div className="min-w-0">
                    <div className="font-semibold text-gray-900">{inv.invoiceNo}</div>
                    <div className="text-sm text-gray-500">Date: {inv.date}</div>
                    {inv.notes ? <div className="text-sm text-gray-500 mt-1">{inv.notes}</div> :
null}

                    {/* Invoice file link */}
                    <div className="mt-2 text-sm">
                      {inv.invoiceFileUrl ? (
                        <a
                          href={BACKEND_URL + inv.invoiceFileUrl}
                          target="_blank"
                          rel="noopener noreferrer"
                          className="text-blue-600 underline font-medium"
                        >
                          View Invoice File
                        </a>
                      ) : (
                        <span className="text-gray-400">No invoice file</span>
                      )}
                    </div>
                  </div>
                </div>

                {/* Right */}
                <div className="flex items-center gap-4 shrink-0">
                  <div className="text-right">
                    <div className="text-lg font-bold
text-gray-900">{formatRwf(Number(inv.amount) || 0)}</div>
                  </div>

                  <span className={`px-3 py-1 rounded text-xs font-semibold
${getInvoiceStatusChip(inv.status)}`}>
                    {inv.status}
```

```
                </span>

                {/* Proof */}
                {inv.status === 'Paid' ? (
                  inv.proofUrl ? (
                    <a
                      href={BACKEND_URL + inv.proofUrl}
                      target="_blank"
                      rel="noopener noreferrer"
                      className="text-blue-600 underline text-xs font-medium"
                    >
                      View Proof
                    </a>
                  ) : (
                    <span className="text-xs text-gray-400">No proof</span>
                  )
                ) : (
                  <button
                    className="px-3 py-1 rounded bg-gray-100 text-gray-700 text-xs font-semibold
hover:bg-gray-200"
                    onClick={() => openUploadProofModal(inv._id!)}
                  >
                    Upload Proof
                  </button>
                )}

                {/* Upload invoice file (if missing) */}
                {!inv.invoiceFileUrl && (
                  <button
                    className="px-3 py-1 rounded bg-gray-100 text-gray-700 text-xs font-semibold
hover:bg-gray-200"
                    onClick={() => {
                      const input = document.createElement('input');
                      input.type = 'file';
                      input.onchange = async () => {
                        const file = input.files?.[0];
                        if (!file) return;
                        try {
                          setInvoicesError('');
                          await uploadInvoiceFile(inv._id!, file);
                          reloadInvoices();
                        } catch (err: any) {
                          setInvoicesError(err.message || 'Failed to upload invoice file');
                        }
```

```
                    };
                    input.click();
                  }}
                >
                  Upload Invoice File
                </button>
              )}

              {/* Delete */}
              <button
                className="p-2 text-red-700 hover:bg-red-50 rounded"
                title="Delete invoice"
                onClick={() => handleDeleteInvoice(inv._id!)}
              >
                <Trash2 className="w-4 h-4" />
              </button>
            </div>
          </div>
        ))}
      </div>
    )}
  </div>

  {/* Add Invoice Modal */}
  {showAddInvoiceModal && (
    <div className="fixed inset-0 bg-gray-900 bg-opacity-50 flex items-center justify-center
z-50 p-4">
      <div className="bg-white rounded-lg max-w-md w-full p-6">
        <h3 className="text-lg font-semibold text-gray-900 mb-4">Create Invoice</h3>

        <form onSubmit={handleAddInvoice} className="space-y-4">
          <div className="grid grid-cols-2 gap-4">
            <div>
              <label className="block text-sm font-medium text-gray-700 mb-1">Date
*</label>
              <input
                type="date"
                value={newInvoice.date}
                onChange={(e) => setNewInvoice((p) => ({ ...p, date: e.target.value }))}
                required
                className="w-full px-3 py-2 border border-gray-300 rounded"
              />
            </div>
```

```
            <div>
              <label className="block text-sm font-medium text-gray-700 mb-1">Amount
(RWF) *</label>
              <input
                inputMode="numeric"
                value={newInvoice.amount}
                onChange={(e) => setNewInvoice((p) => ({ ...p, amount: e.target.value }))}
                required
                className="w-full px-3 py-2 border border-gray-300 rounded"
                placeholder="100000"
              />
            </div>
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Notes</label>
            <textarea
              value={newInvoice.notes}
              onChange={(e) => setNewInvoice((p) => ({ ...p, notes: e.target.value }))}
              rows={2}
              className="w-full px-3 py-2 border border-gray-300 rounded"
            />
          </div>

          {/* optional invoice file */}
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Invoice File
(optional)</label>
            <input type="file" onChange={(e) => setNewInvoiceFile(e.target.files?.[0] || null)}
className="w-full" />
            <p className="text-xs text-gray-500 mt-1">
              Upload the invoice PDF/scan. Proof of payment is uploaded separately.
            </p>
          </div>

          <div className="flex gap-3 mt-6">
            <button
              type="button"
              onClick={() => {
                setShowAddInvoiceModal(false);
                setNewInvoiceFile(null);
              }}
              className="flex-1 px-4 py-2 border border-gray-300 rounded text-gray-700
hover:bg-gray-50"
```

```
                >
                  Cancel
                </button>
                <button type="submit" className="flex-1 px-4 py-2 bg-gray-900 text-white rounded
hover:bg-gray-800">
                  Create Invoice
                </button>
              </div>
            </form>
          </div>
        </div>
      )}

      {/* Upload Proof Modal */}
      {showUploadProofModal && (
        <div className="fixed inset-0 bg-gray-900 bg-opacity-50 flex items-center justify-center
z-50 p-4">
          <div className="bg-white rounded-lg max-w-md w-full p-6">
            <h3 className="text-lg font-semibold text-gray-900 mb-4">Upload Proof of
Payment</h3>

            <form onSubmit={handleUploadProof} className="space-y-4">
              <input
                type="file"
                required
                onChange={(e) => setProofFile(e.target.files?.[0] || null)}
                className="w-full"
              />

              <div className="flex gap-3 mt-6">
                <button
                  type="button"
                  onClick={() => {
                    setShowUploadProofModal(false);
                    setProofInvoiceId(null);
                    setProofFile(null);
                  }}
                  className="flex-1 px-4 py-2 border border-gray-300 rounded text-gray-700
hover:bg-gray-50"
                >
                  Cancel
                </button>
                <button
                  type="submit"
```

```
                disabled={uploadingProof}
                className="flex-1 px-4 py-2 bg-gray-900 text-white rounded hover:bg-gray-800
disabled:opacity-60"
              >
                {uploadingProof ? 'Uploading...' : 'Upload'}
              </button>
            </div>
          </form>
        </div>
      </div>
    )}
  </div>
)}

{/* Audit */}
{activeTab === 'audit' && (
  <div className="bg-white border border-gray-200 rounded-lg overflow-hidden">
    <div className="px-6 py-5 border-b border-gray-200 flex items-center justify-between">
      <div>
        <h2 className="font-semibold text-gray-900">Audit Log</h2>
      </div>
    </div>

    {auditError && (
      <div className="px-6 py-4 text-sm text-red-700 bg-red-50 border-b
border-red-100">{auditError}</div>
    )}

    {auditLoading ? (
      <div className="px-6 py-10 text-gray-500">Loading audit log...</div>
    ) : auditLogs.length === 0 ? (
      <div className="px-6 py-10 text-gray-500">No audit activity yet for this case.</div>
    ) : (
      <div className="divide-y divide-gray-100">
        {auditLogs.map((log) => (
          <div key={log._id} className="px-6 py-5 flex items-start justify-between gap-6">
            <div className="min-w-0">
              <div className="text-sm text-gray-900">
                <span className="font-semibold">{log.actorName}</span>{' '}
                <span className="text-gray-700">{log.message}</span>
              </div>

              {log.detail && <div className="text-sm text-gray-500 mt-1
break-words">{log.detail}</div>}
```

```
                <div className="text-xs text-gray-400 mt-2">{log.action}</div>
              </div>

              <div className="text-xs text-gray-400 whitespace-nowrap">
                {new Date(log.createdAt).toLocaleString()}
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
  )}

  {/* Edit Case Modal (MD/Exec only) */}
  {showEditCase && editCaseData && canManageCase && (
    <div className="fixed inset-0 bg-gray-900 bg-opacity-50 flex items-center justify-center
z-50 p-4">
      <div className="bg-white rounded-lg max-w-lg w-full flex flex-col" style={{ maxHeight:
'90vh' }}>
        <div className="p-6 border-b">
          <h3 className="text-lg font-semibold text-gray-900">Edit Case</h3>
        </div>

        <form onSubmit={handleEditCase} className="flex-1 overflow-y-auto space-y-4 p-6">
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Case No.</label>
            <input
              type="text"
              value={editCaseData.caseNo}
              onChange={(e) => setEditCaseData((c) => (c ? { ...c, caseNo: e.target.value } : c))}
              required
              className="w-full px-3 py-2 border border-gray-300 rounded"
            />
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Parties</label>
            <input
              type="text"
              value={editCaseData.parties}
              onChange={(e) => setEditCaseData((c) => (c ? { ...c, parties: e.target.value } : c))}
              required
              className="w-full px-3 py-2 border border-gray-300 rounded"
            />
```

```
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Case
Type</label>
            <input
              type="text"
              value={editCaseData.caseType}
              onChange={(e) => setEditCaseData((c) => (c ? { ...c, caseType: e.target.value } : c))}
              required
              className="w-full px-3 py-2 border border-gray-300 rounded"
            />
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Assigned
To</label>
            <select
              value={editCaseData.assignedTo}
              onChange={(e) => setEditCaseData((c) => (c ? { ...c, assignedTo: e.target.value } :
c))}
              className="w-full px-3 py-2 border border-gray-300 rounded"
              disabled={staffLoading}
              required
            >
              <option value="">{staffLoading ? 'Loading staff...' : 'Select staff'}</option>
              {staffUsers.map((u) => (
                <option key={u._id} value={u.name}>
                  {u.name}
                </option>
              ))}
            </select>

            {staffError && <p className="text-xs text-red-600 mt-2">{staffError}</p>}
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Priority</label>
            <select
              value={editCaseData.priority}
              onChange={(e) => setEditCaseData((c) => (c ? { ...c, priority: e.target.value } : c))}
              className="w-full px-3 py-2 border border-gray-300 rounded"
            >
              <option value="High">High</option>
```

```
            <option value="Medium">Medium</option>
            <option value="Low">Low</option>
          </select>
        </div>

        <div>
          <label className="block text-sm font-medium text-gray-700 mb-1">Status</label>
          <select
            value={editCaseData.status}
            onChange={(e) => setEditCaseData((c) => (c ? { ...c, status: e.target.value } : c))}
            className="w-full px-3 py-2 border border-gray-300 rounded"
          >
            {STAGE_ORDER.map((status) => (
              <option key={status} value={status}>
                {status}
              </option>
            ))}
          </select>
        </div>

        <div>
          <label className="block text-sm font-medium text-gray-700 mb-1">Budget
(RWF)</label>
          <input
            value={String(editCaseData.budget ?? '')}
            onChange={(e) => setEditCaseData((c) => (c ? { ...c, budget: e.target.value } : c))}
            className="w-full px-3 py-2 border border-gray-300 rounded"
            placeholder="1000000"
          />
        </div>

        <div>
          <label className="block text-sm font-medium text-gray-700
mb-1">Description</label>
          <textarea
            value={editCaseData.description}
            onChange={(e) => setEditCaseData((c) => (c ? { ...c, description: e.target.value } :
c))}
            rows={2}
            className="w-full px-3 py-2 border border-gray-300 rounded"
          />
        </div>

        <div className="flex gap-3 mt-6 sticky bottom-0 bg-white pt-4">
```

```
              <button
                type="button"
                onClick={() => setShowEditCase(false)}
                className="flex-1 px-4 py-2 border border-gray-300 rounded text-gray-700
hover:bg-gray-50"
              >
                Cancel
              </button>
              <button type="submit" className="flex-1 px-4 py-2 bg-gray-800 text-white rounded
hover:bg-gray-700">
                Save Changes
              </button>
            </div>
          </form>
        </div>
      </div>
    )}
  </div>
  );
};

function ArrowDownIcon() {
  return (
    <svg width="16" height="16" viewBox="0 0 24 24" fill="none" aria-hidden="true">
      <path
        d="M12 3v10m0 0 4-4m-4 4-4-4M5 17v3h14v-3"
        stroke="currentColor"
        strokeWidth="2"
        strokeLinecap="round"
        strokeLinejoin="round"
      />
    </svg>
  );
}

export default CaseWorkspace;
```